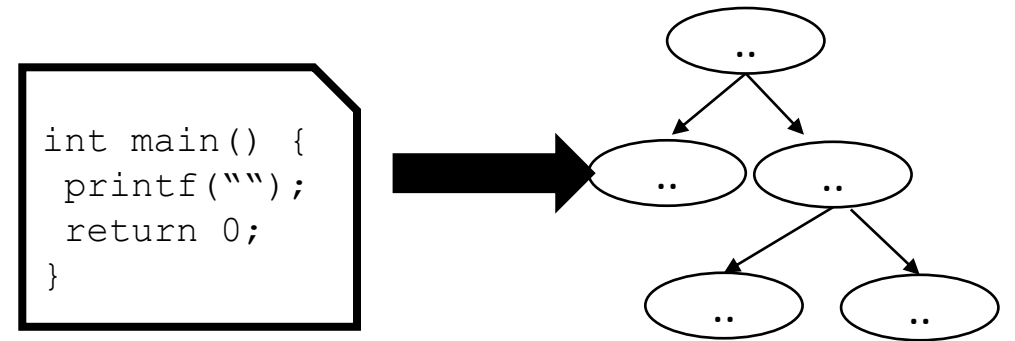


CSE110A: Compilers

Topics: Algorithms for Parsing

- *Syntactic Analysis continued*
 - *Top down parsing*
 - *Rewriting to avoid left recursion*
 - *Oracle Parsing*
 - *LL(1) Parser*
 - *A Recursive Descent Parser*



ORACULARITY:
MAKING A BACKTRACK FREE LL(1) PARSER
USING FIRST, FOLLOW and FIRST+ SETS

New topic: Algorithms for parsing

One goal:

- Given a string s and a CFG G , determine if G can derive s
- We will do that by implicitly attempting to derive a parse tree for S
- Two different approaches, each with different trade-offs:
 - Top down
 - Bottom up

Top-down parsing

input: 2+3+4

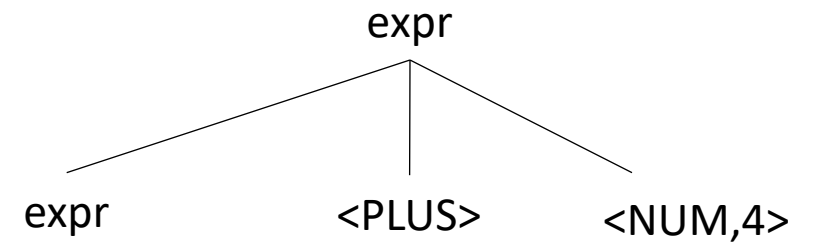
expr

Operator	Name	Productions
+	expr	: expr PLUS NUM NUM

Top-down parsing

input: 2+3+4

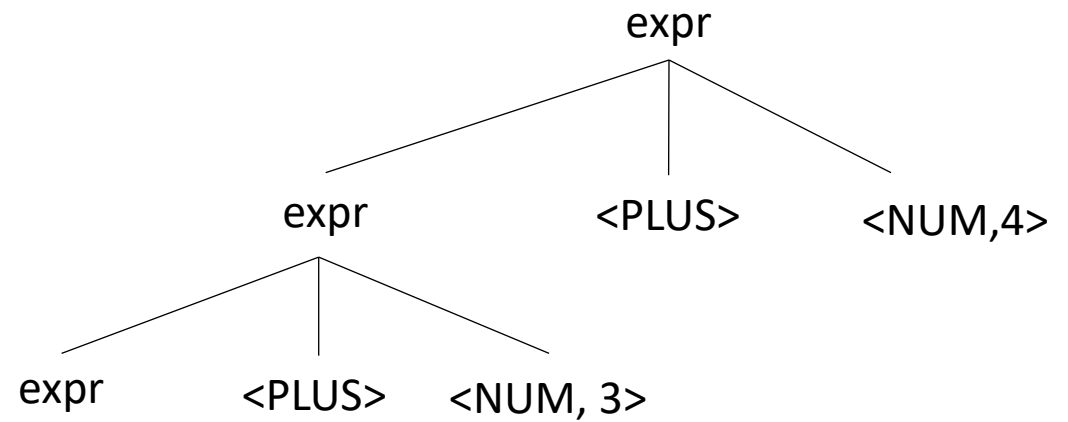
Operator	Name	Productions
+	expr	: expr PLUS NUM NUM



Top-down parsing

input: 2+3+4

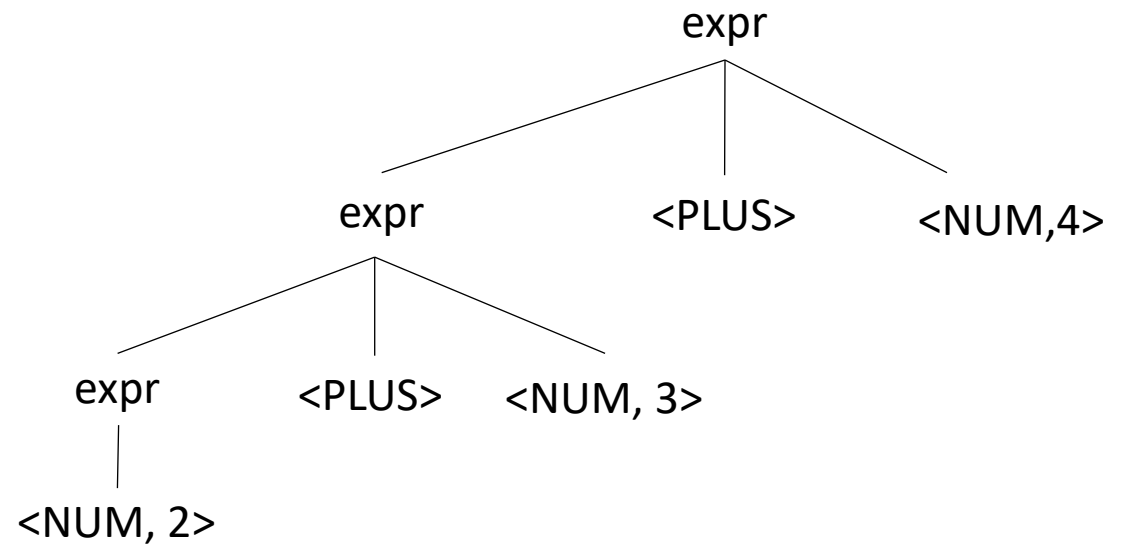
Operator	Name	Productions
+	expr	: expr PLUS NUM NUM



Top-down parsing

input: 2+3+4

Operator	Name	Productions
+	expr	: expr PLUS NUM NUM



Top-down parsing

Pros:

- Algorithm is simpler
- Oftentimes faster than bottom-up
- Better Error Messages

Cons:

- Not efficient on arbitrary grammars
- Many grammars need to be re-written
- More effective error recovery

Bottom-up parsing

input: 2+3+4

Operator	Name	Productions
+	expr	: expr PLUS NUM NUM

<NUM, 2> <PLUS> <NUM, 3> <PLUS> <NUM,4>

Bottom-up parsing

input: 2+3+4

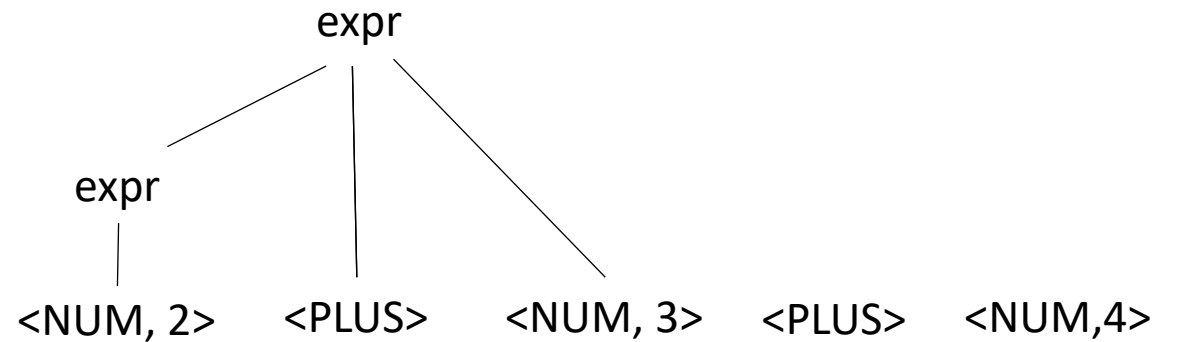
Operator	Name	Productions
+	expr	: expr PLUS NUM NUM

expr
|
<NUM, 2> <PLUS> <NUM, 3> <PLUS> <NUM,4>

Bottom-up parsing

input: 2+3+4

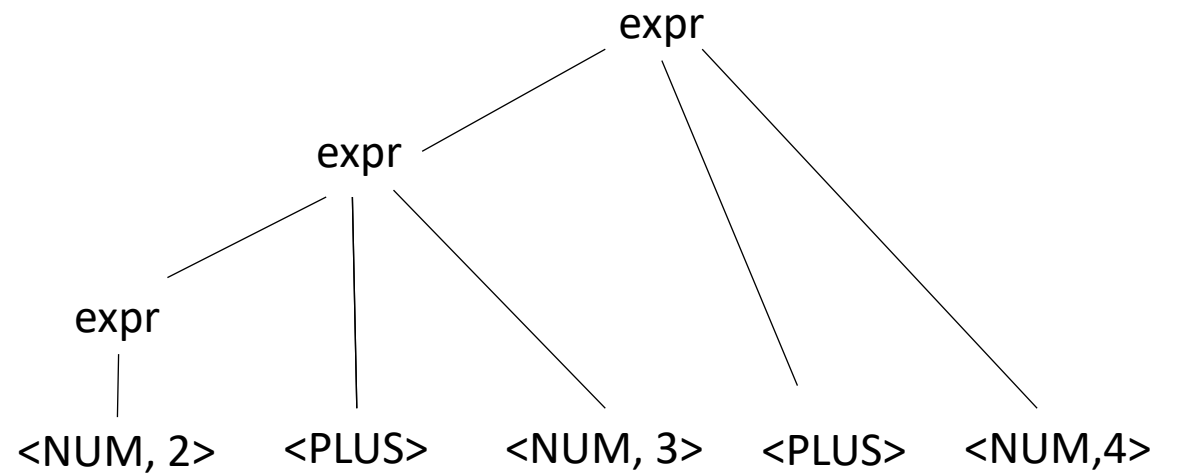
Operator	Name	Productions
+	expr	: expr PLUS NUM NUM



Bottom-up parsing

input: 2+3+4

Operator	Name	Productions
+	expr	: expr PLUS NUM NUM



Bottom up

Pros:

- can handle grammars expressed more naturally
- can encode precedence and associativity even if grammar is ambiguous (with precedence and associativity specifications)

Cons:

- algorithm is complicated
- In some cases slower than top down

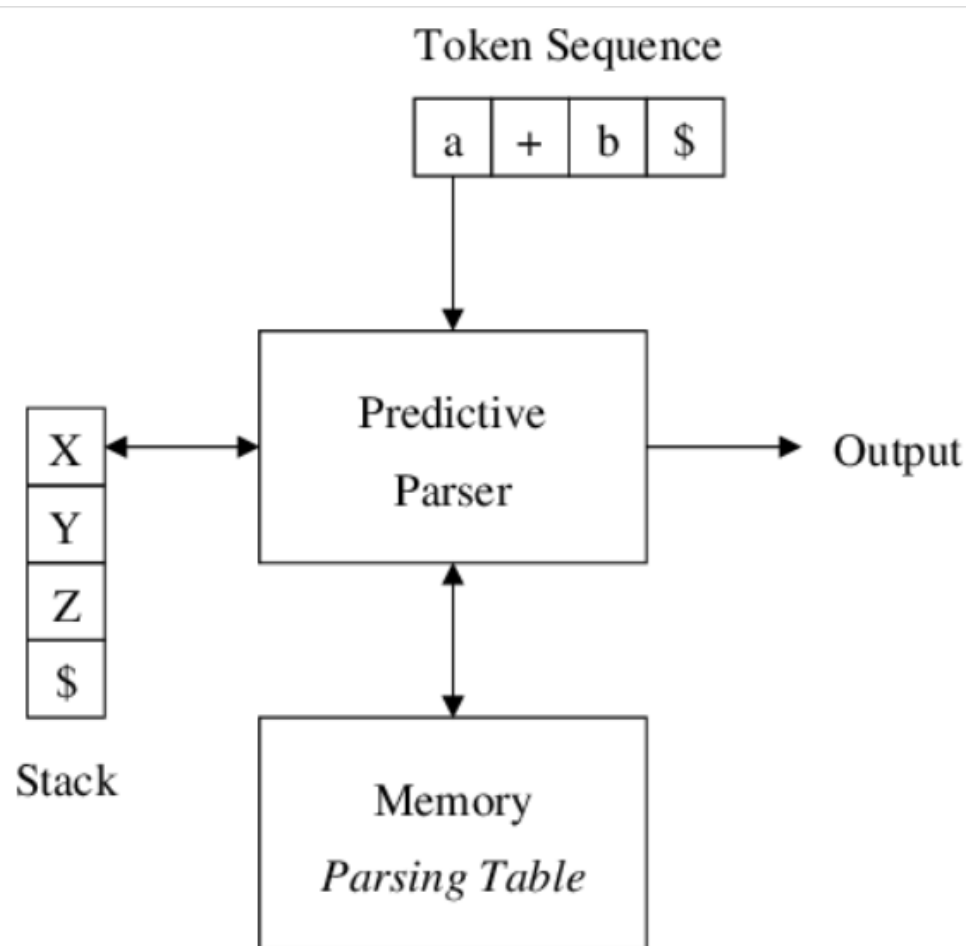
Let's start with top-down parsing:

Two ways to implement the same idea:

- 1) LL(1) Push Down Automaton
- 2) Recursive Descent

Top down, Table Driven: LL(1) PDA Parser

Left to right processing with Leftmost derivation



Expand (Predict)

If top of stack is **non-terminal A**

And lookahead token is **a**

Consult table $M[A, a]$

Replace A in dysvk with chodr RHS
production (pushed in reverse order)

Match (Consume)

If top of stack is a **terminal a**

And lookahead token is also **a**

Pop stack

Advance input

Accept (or Error)

If top of stack is \$

And input token is \$

Parsing succeeds: Accept

Otherwise:

No table entry → **error**

3 Possible Actions

```
root = start symbol;      # Expr
focus = root;
push(None);
to match = s.token();    # Read first token
```

```
while (true): // Expand(Predict)
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1    # First symbol in rule

    else if (focus == to_match): // Match(Consume)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	[]

TOS ←

```

1: Expr ::= Expr Op Unit
2:       | Unit
3: Unit  ::= '(' Expr ')'
4:       | ID
5: Op    ::= '+'
6:       | '*'

```

Can we derive the string $(a+b)^*c$

[illegible]

```
root = start symbol;    # Expr
focus = root;
push(None);
to_match = s.token();    # Read first token
```

```
while (true):
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1    # First symbol in rule

    else if (focus == to_match):
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Initialized State

Variable	Value
focus	Expr
to_match	"("
s.istring	"a+b) *c"
stack	[]

TOS ←

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

Can we derive the string $(a+b) *c$

Expanded Rule	Sentential Form
start	Expr

Initialize

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
while (true):
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Variable	Value
focus	Expr -> Expr
to_match	"("
s.istring	"a+b) *c"
stack	[Op Unit]

TOS ←

- 1: Expr ::= Expr Op Unit
- 2: | Unit
- 3: Unit ::= '(' Expr ')'
- 4: | ID
- 5: Op ::= '+'
- 6: | '*'

Can we derive the string (a+b) *c

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit

1st Iteration: Apply rule 1

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
while (true):
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1                // B1 is now '('

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

Can we derive the string (a+b) *c

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit

Variable	Value
focus	Expr -> Unit
to_match	"("
s.istring	"a+b) *c"
stack	[Op Unit]

TOS ←

2nd Iteration: Apply rule 2

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
while (true):
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1           // B1 is now '('

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

Can we derive the string (a+b) *c

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit
3	'(' Expr ')' Op Unit

Variable	Value
focus	Unit -> "("
to_match	'('
's.istring	+b) *c
Stack	[Expr ')' Op Unit]

TOS ←

3rd Iteration: gobble the "("

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

Currently we assume there is some magic that we can pick the right rule every time from production with LHS of A

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

```
while (true):
    if (focus is a nonterminal, A)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1           // B1 is now '('
```

```
else if (focus == to_match)
    to_match = s.token()
    focus = pop()
```

```
else if (to_match == None and focus == None)
    Accept
```

Can we derive the string (a+b) *c

Expanded Rule	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit
3	Expr ')' Op Unit

Variable	Value
focus	"(" -> Expr
to_match	(' -> (ID, "a")
's.istring	+b) *c
Stack	[') Op Unit]

TOS ←

4th Iteration: consume "("

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();

while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Variable	Value
focus	Op
to_match	'+'
s.istring	b) *c
stack	Unit ')' Op, Expr, None

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

Can we derive the string (a+b) *c

Expanded Rule #	Sentential Form
start	Expr
1	Expr Op Unit
2	Unit Op Unit
3	'(' Expr ')' Op Unit
1	'(' Expr Op Unit ')' Op Unit
2	'(' Unit Op Unit ')' Op Unit
4	'(' ID Op Unit ')' Op Unit

And so on...

BUT

What can go wrong if we don't have a magic Choice?

```

1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit  ::= '(' Expr ')'
4:      | ID
5: Op    ::= '+'
6:      | '*'

```

Can we derive the string $(a+b)^*c$

[illegible]

Variable	Value
focus	
to_match	
s.istring	
stack	

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

What can go wrong?

```
while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

Can we derive the string (a+b) *c

Expanded Rule	Sentential Form
start	Expr
2	Expr Op Unit
2	Expr Op Unit Op Unit
2	Expr Op Unit Op Unit Op Unit
2	Expr Op Unit

Infinite recursion, that is what!

Top down parsing does not handle left recursion

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit ::= '(' Expr ')'
4:      | ID
5: Op  ::= '+'
6:      | '*'
```

direct left recursion

```
1: Expr_base ::= Unit
2:          | Expr_op
3: Expr_op  ::= Expr_base Op Unit
4: Unit     ::= '(' Expr_base ')'
5:          | ID
6: Op       ::= '+'
7:          | '*'
```

indirect left recursion

Top down parsing cannot handle either of these

Top down parsing does not handle left recursion

- In general, any CFG can be re-written without left recursion
 - However, the transformation may affect associativity
 - or increase the number of rules
 - but it is always possible

Eliminating direct left recursion

```
Fee ::= Fee "a"  
      |    "b"
```

What does this grammar describe?

Eliminating direct left recursion

The grammar can be rewritten as

$$\begin{array}{l} \text{Fee} ::= \text{Fee } \text{"a"} \\ \quad | \quad \text{"b"} \end{array}$$
$$\text{Fee} ::= \text{"b"} \text{Fee2}$$
$$\begin{array}{l} \text{Fee2} ::= \text{"a"} \text{Fee2} \\ \quad | \quad \text{"\""} \end{array}$$

Eliminating direct left recursion

*In general, **A and B can be any sequence of non-terminals and terminals***

$$\begin{array}{l} \text{Fee} ::= \text{Fee } A \\ \quad | \quad B \end{array}$$
$$\text{Fee} ::= B \text{ Fee2}$$
$$\begin{array}{l} \text{Fee2} ::= A \text{ Fee2} \\ \quad | \quad \text{"\""} \end{array}$$

Eliminating direct left recursion

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit  ::= '(' Expr ')'
4:      | ID
5: Op    ::= '+'
6:      | '*'
```

Lets do this one as an example:

Fee	::=	Fee	A
			B



Fee	::=	B	Fee2
Fee2	::=	A	Fee2
			""

Eliminating direct left recursion

A = ?
B = ?

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit  ::= '(' Expr ')'
4:      | ID
5: Op    ::= '+'
6:      | '*'
```

```
1: Expr    ::= ?
2: Expr2   ::= ?
3:         | ?
4: Unit    ::= '(' Expr ')'
5:         | ID
6: Op      ::= '+'
7:         | '*'
```

Lets do this one as an example:

```
Fee ::= Fee A
     | B
```



```
Fee  ::= B Fee2
Fee2 ::= A Fee2
      | ""
```

Eliminating direct left recursion

A = Op Unit
B = Unit

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit  ::= '(' Expr ')'
4:      | ID
5: Op    ::= '+'
6:      | '*'
```

```
1: Expr    ::= ?
2: Expr2   ::= ?
3:         | ?
4: Unit    ::= '(' Expr ')'
5:         | ID
6: Op      ::= '+'
7:         | '*'
```

Lets do this one as an example:

```
Fee ::= Fee A
     | B
```



```
Fee  ::= B Fee2
Fee2 ::= A Fee2
      | ""
```

Eliminating direct left recursion

A = Op Unit
B = Unit

```
1: Expr ::= Expr Op Unit
2:      | Unit
3: Unit  ::= '(' Expr ')'
4:      | ID
5: Op    ::= '+'
6:      | '*'
```

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

Lets do this one as an example:

```
Fee ::= Fee A
     | B
```



```
Fee  ::= B Fee2
Fee2 ::= A Fee2
     | ""
```

```
while (true):
    if (focus is a nonterminal)
        pick next rule ( $A ::= B_1, B_2, B_3 \dots B_N$ );
        push( $B_N \dots B_3, B_2$ );
        focus =  $B_1$ 

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

```

1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:         | ""
4: Unit ::= '(' Expr ')'
5:         | ID
6: Op ::= '+'
7:         | '*'

```

[illegible]

```
root = start symbol;
focus = root;
push(None);
to match = s.token();
```

```
while (true):  
    if (focus is a nonterminal)  
        pick next rule ( $A ::= B_1, B_2, B_3 \dots B_N$ );  
        push( $B_N \dots B_3, B_2$ );  
        focus =  $B_1$ 
```

```

else if (focus == to_match)
    to_match = s.token()
    focus = pop()

```

```
else if (to_match == None and focus == None)
    Accept
```

Variable	Value
focus	
to_match	
s.istring	
stack	

*How to handle
this case?*

```

1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit ::= '(' Expr ')'
5:      | ID
6: Op ::= '+'
7:      | '*'

```

[illegible]

```
while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        if A == "": focus=pop(); continue; # ignore it
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

```

1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit ::= '(' Expr ')'
5:      | ID
6: Op ::= '+'
7:      | '*'

```

[illegible]

How about indirect left recursion?

```
1: Expr ::= Expr Op Unit
2:       | Unit
3: Unit  ::= '(' Expr ')'
4:       | ID
5: Op    ::= '+'
6:       | '*'
```

direct left recursion

```
1: Expr_base ::= Unit
2:           | Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

indirect left recursion

Top down parsing cannot handle either

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           | Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

Identify indirect left left recursion

$$\text{Expr_base} \rightarrow_{lhs} \text{Expr_op} \rightarrow_{lhs} \text{Expr_base}$$

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           | Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

Identify indirect left left recursion

$Expr_base \rightarrow_{lhs} Expr_op \rightarrow_{lhs} Expr_base$

Substitute indirect non-terminal closer to initial non-terminal

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           | Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

```
1: Expr_base ::= Unit
2:           | Expr_base Op Unit
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

Identify indirect left left recursion

What to do with production rule 3?

$Expr_base \rightarrow_{lhs} Expr_op \rightarrow_{lhs} Expr_base$

Substitute indirect non-terminal closer to initial non-terminal

How about indirect left recursion?

```
1: Expr_base ::= Unit
2:           | Expr_op
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

```
1: Expr_base ::= Unit
2:           | Expr_base Op Unit
3: Expr_op   ::= Expr_base Op Unit
4: Unit      ::= '(' Expr_base ')'
5:           | ID
6: Op        ::= '+'
7:           | '*'
```

Identify indirect left left recursion

What to do with production rule 3?

It may need to stay if another production rule references it!

$Expr_base \rightarrow_{lhs} Expr_op \rightarrow_{lhs} Expr_base$

Substitute indirect non-terminal closer to initial non-terminal

It is always possible to eliminate left recursion

Back Tracking

(not a good thing)

```
root = start symbol;  
focus = root;  
push(None);  
to_match = s.token();
```

```
while (true):  
    if (focus is a nonterminal)  
        cache_state();  
        pick next rule (A ::= B1,B2,B3...BN);  
        if B1 == "": focus=pop(); continue;  
        push(BN... B3, B2);  
        focus = B1  
  
    else if (to_match == None and focus == None)  
        Accept  
  
    else if (focus == to_match)  
        to_match = s.token()  
        focus = pop()  
  
    else if (we have a cached state)  
        backtrack();  
  
    else  
        parser_error()
```

*Keep track of what
choices we've made*

```
1: Expr ::= ID Expr2  
2: Expr2 ::= '+' Expr2  
           |      ""
```

Can we match: “a”?

Expanded Rule	Sentential Form
start	Expr
1	ID Expr2
3	ID

Backtracking gets complicated...

- Do we need to backtrack?
 - In the general case, **yes**
 - In many useful cases, **no**

```
root = start symbol;
focus = root;
push(None);
to_match = s.token();
```

```
while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        if B1 == "": focus=pop(); continue;
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept
```

Could we make a smarter choice here?

```
1: Expr ::= ID Expr2
2: Expr2 ::= '+' Expr2
3:         | ""
```

Can we match: "a"?

Variable	Value
focus	Expr2
to_match	None
s.istring	""
stack	None

Expanded Rule	Sentential Form
start	Expr
1	ID Expr2

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First sets:

```
1: {}
2: {}
3: {}
4: {}
5: {}
6: {}
7: {}
```

Example

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       |  ""
4: Unit  ::= '(' Expr ')'
5:       |  ID
6: Op    ::= '+'
7:       |  '*'
```

First sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { "" }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

The First Set

For each production choice, find the set of tokens that each production can start with

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      |  ""
4: Unit  ::= '(' Expr ')'
5:      |  ID
6: Op    ::= '+'
7:      |  '*'
```

First sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { "" }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

We can use first sets to decide which rule to pick!

```

root = start symbol;
focus = root;
push(None);
to_match = s.token();

```

```

while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept

```



```

1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'

```

First sets:

```

1: {'(', ID}
2: {'+', '*'}
3: {''}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}

```

Look-ahead to select
Correct rule



We simply use to_match and compare it
to the first sets for each choice

For example, Op and Unit

Variable

Value

focus

to_match

s.istring

stack

The Follow Set

Rules with “” in their First set need special attention

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | ""
4: Unit  ::= '(' Expr ')'
5:       | ID
6: Op    ::= '+'
7:       | '*'
```

First sets:	Follow sets:
1: { '(', ID }	1: NA
2: { '+', '*' }	2: NA
3: { "" }	3: { }
4: { '(' }	4: NA
5: { ID }	5: NA
6: { '+' }	6: NA
7: { '*' }	7: NA

We need to find the tokens that any string that follows the production can start with.

The Follow Set

Rules with “” in their First set need special attention

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First sets:

```
1: {'(', ID}
2: {'+', '*'}
3: {""}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

Follow sets:

```
1: NA
2: NA
3: {None, ')'}
4: NA
5: NA
6: NA
7: NA
```

We need to find all tokens that any string that follows a production containing $\in$ in its first set can start with. In the example above Expr2 can be followed by what follows Expr or ')'.
Note that FIRST, FOLLOW, and FIRST+ are sets of terminals. But follow uses Non-terminals to calculate with transitive closure.

The First+ Set

The First+ set is conditional combination of First and Follow sets

$$\text{FIRST}^+(A \rightarrow \beta) = \begin{cases} \text{FIRST}(\beta) & \text{if } \epsilon \in \text{FIRST}(\beta) \\ \text{FIRST}(\beta) \cup \text{FOLLOW}(A) & \text{otherwise} \end{cases}$$

where $\beta = X_1X_2 \dots X_n$ is a sequence of terminals/non-terminals

We only apply FOLLOW(A) if FIRST(A $\rightarrow$ β) has ϵ as a member.

This is pretty subtle

The First+ Set

The First+ set is the combination of First and Follow sets

	First sets:	Follow sets:	First+ sets:
1: Expr ::= Unit Expr2	1: { '(' , ID }	1: NA	1: { '(' , ID }
2: Expr2 ::= Op Unit Expr2	2: { '+' , '*' }	2: NA	2: { '+' , '*' }
3: ""	3: { "" }	3: { None , ')' }	3: { None , ')' }
4: Unit ::= '(' Expr ')'	4: { '(' }	4: NA	4: { '(' }
5: ID	5: { ID }	5: NA	5: { ID }
6: Op ::= '+'	6: { '+' }	6: NA	6: { '+' }
7: '*'	7: { '*' }	7: NA	7: { '*' }

Animation of First, Follow, and First+ Sets

<https://youtu.be/JhEmguMmfLU>

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit ::= '(' Expr ')'
5:      | ID
6: Op ::= '+'
7:      | '*'
```

First+ sets:

```
1: {'(', ID}
2: {'+', '*'}
3: {None, ')'}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

For each non-terminal: if every production has a disjoint First+ set then we do not need any backtracking!

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit ::= '(' Expr ')'
5:      | ID
6: Op ::= '+'
7:      | '*'
```

First+ sets:

```
1: {'(', ID}
2: {'+', '*'}
3: {None, ')'}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

For each non-terminal: if every production has a disjoint First+ set then we do not need any backtracking!

Do we need backtracking?

The First+ set is the combination of First and Follow sets

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

These grammars are called LL(1)

- L - scanning the input left to right
- L - left derivation
- 1 - how many look ahead symbols

They are also called predictive grammars

Many programming languages are LL(1)

None implies the end of the program.

*For each non-terminal: if every production has a **disjoint First+ set** then we do not need any backtracking!*

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID
2:         | ID '[' Args ']'      # Array Indexing
3:         | ID '(' Args ')'      # function calling
...
```

Sometimes the grammar needs to be refactored

1: Factor ::= ID	First
2: ID '[' Args ']'	1: {}
3: ID '(' Args ')'	2: {}
...	3: {}
	...

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID
2:         | ID '[' Args ']'
3:         | ID '(' Args ')'
...
```

First

```
1: {ID}
2: {ID}
3: {ID}
...
```

We cannot select the next rule based on a single look ahead token!

Sometimes the grammar needs to be refactored

		First
1:	Factor ::= ID	1: {ID}
2:	ID '[' Args ']'	2: {ID}
3:	ID '(' Args ')'	3: {ID}
...		...

We can refactor

		First
1:	Factor ::= ID Option_args	1: {}
2:	Option_args ::= '[' Args ']'	2: {}
3:	'(' Args ')'	3: {}
4:	""	4: {}

Sometimes the grammar needs to be refactored

1: Factor ::= ID	First
2: ID '[' Args ']'	1: {ID}
3: ID '(' Args ')'	2: {ID}
...	3: {ID}
	...

We can refactor

1: Factor ::= ID Option_args	First
2: Option_args ::= '[' Args ']'	1: {ID}
3: '(' Args ')'	2: {'[' ']'}
4: ""	3: {'(' ')'} 4: {""}

// We will need to compute the follow set

Sometimes the grammar needs to be refactored

```
1: Factor ::= ID
2:         | ID '[' Args ']'
3:         | ID '(' Args ')'
...
```

```
First
1: {ID}
2: {ID}
3: {ID}
...
```

It is not always possible to rewrite any grammar into a predictive form, but many programming languages can be.

We can refactor

```
1: Factor      ::= ID Option_args
2: Option_args ::= '[' Args ']'
3:             | '(' Args ')'
4:             | ""
```

```
First
1: {ID}
2: {'['}
3: {'('}
4: {""}
```

*// We will need to compute the follow set
// to deal with the empty string, and get
// First+*

We now have a full top-down parsing algorithm!

```

root = start symbol;
focus = root;
push(None);
to_match = s.token();

```

```

while (true):
    if (focus is a nonterminal)
        pick next rule (A ::= B1,B2,B3...BN);
        push(BN... B3, B2);
        focus = B1

    else if (focus == to_match)
        to_match = s.token()
        focus = pop()

    else if (to_match == None and focus == None)
        Accept

```

```

First+ sets:
1: { '(' , ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }

```

*First+ sets for each
production rule*

```

1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'

```

*input grammar,
refactored to remove
left recursion*

To pick the next rule, compare `to_match` with the possible `first+` sets.
Pick the rule whose `first+` set contains `to_match`.

If there is no such rule then it is a parsing error.

Moving on to a simpler implementation:

Recursive Descent Parser

Let's look at the grammar

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      |  ""
4: Unit  ::= '(' Expr ')'
5:      |  ID
6: Op    ::= '+'
7:      |  '*'
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse an Expr?

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse an Expr?

We parse a Unit followed by an Expr2

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse an Expr?

We parse a Unit followed by an Expr2

We can just write exactly that!

```
def parse_Expr(self):
    self.parse_Unit();
    self.parse_Expr2();
    return
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse an Expr2?

Let's look at the grammar

```
1: Expr  ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:       | ""
4: Unit  ::= '(' Expr ')'
5:       | ID
6: Op    ::= '+'
7:       | '*'
```

How do we parse an Expr2?

First+ sets:

```
1: {'(', ID}
2: {'+', '*'}
3: {None, ')'}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

How do we parse an Expr2?

```
def parse_Expr2(self):
    token_id = get_token_id(self.to_match)

    # Expr2 ::= Op Unit Expr2
    if token_id in ["PLUS", "MULT"]:
        self.parse_Op()
        self.parse_Unit()
        self.parse_Expr2()
        return

    # Expr2 ::= ""
    if token_id in [None, "RPAR"]:
        return
    else:
        raise ParserException(self.linennumber, # line number (for you to do)
                               self.to_match,    # observed token
                               ["PLUS", "MULT", "RPAR"]) # expected token
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse a Unit?

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

How do we parse a Unit?

```
def parse_Unit(self):
```

```
    token_id = get_token_id(self.to_match)
```

```
    # Unit ::= '(' Expr ')'
```

```
    if token_id == "LPAR":
```

```
        self.eat("LPAR")
```

```
        self.parse_Expr()
```

```
        self.eat("RPAR")
```

```
        return
```

```
    # Unit ::= ID
```

```
    if token_id == "ID":
```

```
        self.eat("ID")
```

```
        return
```

```
def eat(self, expected_token):
```

```
    if self.to_match == expected_token:
```

```
        self.advance() # move to the next token
```

```
    else:
```

```
        raise ParserException(
```

```
            self.current_line, # line# (for you to do)
```

```
            self.to_match, # observed token
```

```
            [expected_token]) # i.e. PAR or RPAR
```

Let's look at the grammar

How do we parse a Unit?

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

```
def parse_Unit(self):
```

```
    token_id = get_token_id(self.to_match)
```

```
    # Unit ::= '(' Expr ')'
```

```
    if token_id == "LPAR":
```

```
        self.eat("LPAR")
```

```
        self.parse_Expr()
```

```
        self.eat("RPAR")
```

```
        return
```

```
    # Unit ::= ID
```

```
    if token_id == "ID":
```

```
        self.eat("ID")
```

```
        return
```

Note: function `eat` must ensure that `to_match` has the expected token ID and advances to the next token, i.e. something like this:

```
def eat(self, expected_token):
```

```
    if self.to_match == expected_token:
```

```
        self.advance() # move to the next token
```

```
    else:
```

```
        raise ParserException(
```

```
            self.current_line,      # line# (for you to do)
```

```
            self.to_match,          # observed token
```

```
            [expected_token])      # LPAR or RPAR or ID)
```

```
class ParserException(Exception):
```

```
    def __init__(self, line_number, found_token, expected_tokens):
```

```
        self.line_number = line_number
```

```
        self.found_token = found_token
```

```
        self.expected_tokens = expected_tokens
```

```
        # Create a readable error message
```

```
        message = (f"Parse error on line {line_number}: found '{found_token}', "
```

```
                    f"expected one of {expected_tokens}")
```

```
        super().__init__(message)
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

How do we parse an Op?

First+ sets:

```
1: {'(', ID}
2: {'+', '*'}
3: {None, ')'}
4: {'('}
5: {ID}
6: {'+'}
7: {'*'}
```

Let's look at the grammar

```
1: Expr ::= Unit Expr2
2: Expr2 ::= Op Unit Expr2
3:      | ""
4: Unit  ::= '(' Expr ')'
5:      | ID
6: Op    ::= '+'
7:      | '*'
```

First+ sets:

```
1: { '(', ID }
2: { '+', '*' }
3: { None, ')' }
4: { '(' }
5: { ID }
6: { '+' }
7: { '*' }
```

How do we parse an Op?

```
def parse_Op(self):
```

```
    token_id = get_token_id(self.to_match)
```

```
    # Op ::= '+'
```

```
    if token_id == "PLUS":
```

```
        self.eat("PLUS")
```

```
        return
```

```
    # Op ::= '*'
```

```
    if token_id == "MULT":
```

```
        self.eat("MULT")
```

```
        return
```

```
def eat(self, expected_token):
```

```
    ...
```

```
    raise ParserException(
```

```
        self.current_line,
```

```
        self.to_match,      # observed token
```

```
        [expected_token]) # expected token
```

```
    ...
```

Recursive Descent
IS THAT SIMPLE